

IPVC — ESTG

Engenharia da Computação Gráfica e Multimédia

Engenharia de Software — 2025/2026

ChromaMood

Vibe Coding e CI/CD

Rodrigo Fernandes Malheiro — 33103

Eduardo Oliveira — 33137

Junho de 2026

Índice

Índice	2
1. Introdução.....	3
2. Tema e Descrição da Solução	3
2.1 Descrição da Solução.....	3
2.2 Arquitetura do Processo de Desenvolvimento	3
3. Plataforma Escolhida e Limitações	4
3.1 Ferramentas Utilizadas.....	4
3.2 Exemplos de Prompts Utilizados	4
3.3 Limitações das Plataformas e da Integração CI/CD.....	5
4. GitHub, Agile e CI/CD	5
4.1 GitHub — Versionamento e Colaboração.....	5
4.2 Agile/Scrum — GitHub Projects.....	6
4.3 CI/CD — Pipeline e Deploy Automático	7
5. Prints e Evidências.....	9
5.1 Aplicação em Funcionamento.....	9
5.2 GitHub	9
5.3 Scrum Board	11
5.4 CI/CD e Deploy.....	12
6. Conclusão e Análise Crítica	12
6.1 Processo e Metodologias de Desenvolvimento.....	12
6.2 Potencial das Ferramentas de IA na Geração e Automatização	13
6.3 Limitações Encontradas	13
6.4 Vantagens do Trabalho Colaborativo.....	13
6.5 Impacto do CI/CD no Desenvolvimento	14
7. Bibliografia	14

1. Introdução

O presente relatório descreve o processo de desenvolvimento do projeto ChromaMood, realizado no âmbito da unidade curricular de Engenharia de Software do curso de Engenharia da Computação Gráfica e Multimédia (ECGM) do IPVC/ESTG, no ano letivo de 2025/2026.

O objetivo desta atividade consistiu em demonstrar a aplicação de processos modernos de Engenharia de Software no desenvolvimento colaborativo de soluções digitais, integrando ferramentas de inteligência artificial na prototipagem e apoio ao desenvolvimento, juntamente com práticas de versionamento, Agile/Scrum, CI/CD e deploy automatizado.

O ChromaMood é uma aplicação web que gera paletas de cor, sugestões tipográficas e descrições de mood a partir de uma descrição textual fornecida pelo utilizador, recorrendo a um modelo de linguagem de grande escala (LLM) via API da Groq.

2. Tema e Descrição da Solução

2.1 Descrição da Solução

O ChromaMood permite ao utilizador descrever um ambiente ou mood em linguagem natural — por exemplo, "café vintage à noite" ou "floresta ao amanhecer" — e receber automaticamente uma paleta de 5 cores em formato HEX, clicáveis para copiar o código, sugestões de tipografia para título e corpo de texto, e uma descrição evocativa do mood interpretado pela IA.

O valor gerado pela solução reside na democratização do processo criativo de design, permitindo que qualquer utilizador, independentemente do seu conhecimento técnico, obtenha uma identidade visual coerente a partir de uma simples descrição textual. A aplicação é igualmente útil para designers que pretendam explorar rapidamente diferentes direções criativas.

2.2 Arquitetura do Processo de Desenvolvimento

A aplicação segue uma arquitetura cliente-servidor simplificada, adequada ao ambiente de deploy serverless:

- Frontend: HTML, CSS e JavaScript puros, sem frameworks, garantindo leveza e compatibilidade universal
- Backend: Função serverless em Node.js (api/generate.js), executada no ambiente Vercel, responsável por comunicar com a API da Groq
- IA: Modelo llama-3.3-70b-versatile via Groq API, que recebe o prompt do utilizador e devolve um objeto JSON estruturado com cores, tipografia e mood

- Deploy: Vercel, com integração direta ao repositório GitHub para deploy automático a cada push para main
- CI/CD: GitHub Actions, com pipeline de validação de ficheiros, lint HTML, unit tests e verificação de sintaxe JavaScript

3. Plataforma Escolhida e Limitações

3.1 Ferramentas Utilizadas

Ferramenta	Função	Vantagens	Dificuldades
Claude (Anthropic)	Geração de código (vibe coding)	Gera código funcional e bem estruturado; explica cada passo; sugere boas práticas	Necessidade de prompts iterativos para afinar detalhes
Google Gemini	Apoio ao desenvolvimento e pesquisa	Bom para pesquisa rápida e explicações conceituais	Respostas menos precisas para código específico
Groq API	Inferência LLM em runtime	Extremamente rápida; gratuita para uso académico; suporta modelos open-source	Respostas ocasionalmente com markdown no JSON; parsing defensivo necessário
Vercel	Deploy e hosting	Deploy automático a cada push; integração nativa com GitHub; suporta serverless functions	Limite de 10s de execução das funções serverless no plano gratuito
GitHub Actions	Pipeline CI/CD	Gratuito para repositórios públicos; integrado no GitHub; fácil de configurar	Curva de aprendizagem inicial na sintaxe YAML
GitHub Projects	Gestão Agile/Scrum	Integrado com issues e PRs; gratuito	Menos funcionalidades que Jira para Scrum formal

3.2 Exemplos de Prompts Utilizados

Durante o desenvolvimento foram utilizados prompts com o Claude e o Google Gemini para geração e refinamento do código. Seguem-se exemplos representativos:

Prompt 1 — Estrutura base do projeto:

Cria uma aplicação web simples em HTML, CSS e JavaScript que permita ao utilizador escrever uma descrição de um mood e gerar uma paleta de 5 cores. O backend deve ser uma serverless function no Vercel que chama a API da Groq com o modelo llama e devolve um JSON com colors, typography e mood.

Prompt 2 — Correção do parsing da resposta:

A API da Groq está a devolver a resposta com backticks de markdown à volta do JSON. Corrige o parsing no backend para limpar esse conteúdo antes de fazer `JSON.parse`.

Prompt 3 — Geração de unit tests:

Gera unit tests com Jest para validar: cores HEX válidas/inválidas, estrutura da resposta da API, validação do prompt do utilizador e mock da chamada fetch ao endpoint `/api/generate`.

Prompt 4 — GitHub Actions:

Cria um workflow de GitHub Actions para este projeto que verifique a presença dos ficheiros essenciais, faça lint ao HTML com `htmlhint`, corra os unit tests com Jest e verifique a sintaxe do JavaScript.

3.3 Limitações das Plataformas e da Integração CI/CD

A principal limitação técnica encontrada foi a inconsistência do formato da resposta da Groq — o modelo instruído a responder apenas em JSON incluía ocasionalmente backticks de markdown, exigindo limpeza defensiva da resposta. Esta situação ilustra um desafio recorrente na integração de LLMs em aplicações produtivas.

Relativamente ao modelo de geração de paletas, verificou-se que o `llama-3.3-70b-versatile` nem sempre produz paletas visualmente coerentes com o mood descrito, tendendo a gerar cores sem relação emocional clara entre si. Este é um problema inerente à natureza probabilística dos LLMs e às limitações do modelo na interpretação de conceitos de design.

O plano gratuito do Vercel impõe um limite de 10 segundos de execução nas funções serverless, o que pode ser insuficiente em casos de elevada latência da Groq. O pipeline CI/CD não inclui testes de integração end-to-end, uma vez que a execução da função serverless requer a chave de API da Groq, indisponível no ambiente de CI sem configuração adicional de secrets.

4. GitHub, Agile e CI/CD

4.1 GitHub — Versionamento e Colaboração

O repositório encontra-se disponível em: github.com/rodrigo09malheiro/chromamood

O projeto utilizou uma estratégia de branching baseada em feature branches, com integração via Pull Requests para a branch main. As branches criadas foram as seguintes:

- feature/estrutura-base — estrutura inicial HTML/CSS/JS (Rodrigo)
- feature/integracao-groq — serverless function e integração com API Groq (Rodrigo)
- feature/cicd-vercel — configuração do Vercel e deploy automático (Rodrigo)
- feat/ui-form-swatches-3 — formulário, swatches, localStorage e polimento visual (Eduardo)

Foram realizados 14 Pull Requests e mais de 20 commits ao longo do projeto, com participação ativa dos dois elementos do grupo.



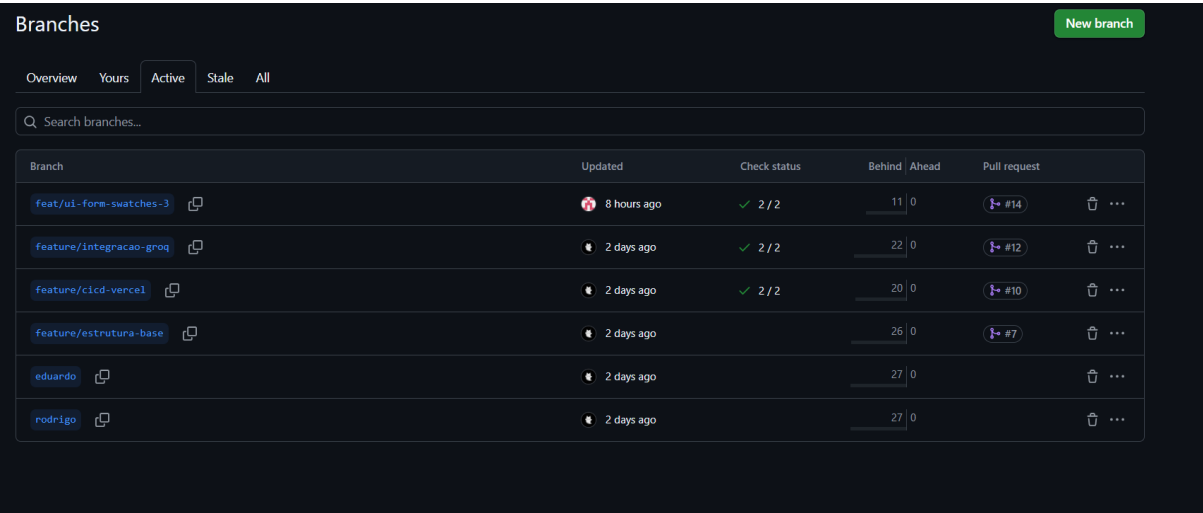
Commits

main

Commits on Jun 18, 2026

revert: voltar ao llama-3.3-70b com prompt few-shot	a7844ce
feat: paletas baseadas em categorias emocionais para consistência visual	641088f
fix: trocar modelo Groq para gemma2-9b-it para melhor coerência de paletas	22d6771
feat: melhorar prompt com few-shot examples para paletas coerentes	acaccf4
feat: melhorar prompt do sistema para paletas mais coerentes	e474ad9
docs: atualizar README com descrição, links e estrutura do projeto	02b73a4
test: adicionar unit tests com Jest e integrar no CI	22d1b23

Figure 1 - Lista de Commits



Branches

Overview Yours Active Stale All

Search branches...

Branch	Updated	Check status	Behind	Ahead	Pull request
feat/ui-form-swatches-3	8 hours ago	2 / 2	11	0	#14
feature/integracao-groq	2 days ago	2 / 2	22	0	#12
feature/cicd-vercel	2 days ago	2 / 2	20	0	#10
feature/estrutura-base	2 days ago		26	0	#7
eduardo	2 days ago		27	0	
rodrigo	2 days ago		27	0	

Figure 2 - Lista de Branches

4.2 Agile/Scrum — GitHub Projects

A gestão do projeto foi realizada com o GitHub Projects, utilizando um Scrum Board com cinco colunas: Product Backlog, Sprint 1, Em Progresso, Em Revisão e Conclusão. A Sprint 1 incluiu as seguintes tarefas:

Issue	Título	Responsável	Estado
#1	[FEAT] Estrutura base do projeto	Rodrigo	Concluído
#2	[FEAT] Integração com API Groq	Rodrigo	Concluído
#3	[FEAT] UI — Formulário e swatches de cor	Eduardo	Concluído
#4	[FEAT] Histórico de paletas (localStorage)	Eduardo	Concluído
#5	[FEAT] Pipeline CI/CD e deploy no Vercel	Rodrigo	Concluído
#15	[DOCS] Relatório final do projeto	Rodrigo	Em Progresso
#16	[TEST] Unit tests com Jest	Rodrigo	Em Revisão
#17	[DOCS] Atualizar README	Rodrigo	Sprint 1

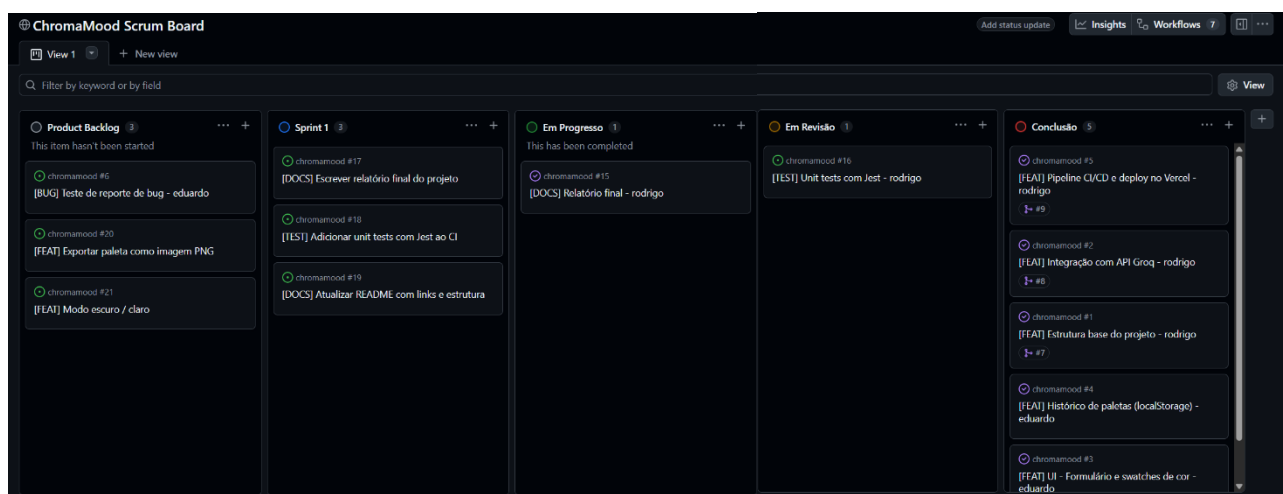


Figure 3 - Quadro Scrum do repositório

4.3 CI/CD — Pipeline e Deploy Automático

O pipeline de CI/CD foi implementado com GitHub Actions (ficheiro `.github/workflows/ci.yml`) e integrado com o Vercel para deploy automático. O fluxo completo é o seguinte:

1. Trigger: push ou pull request para a branch main
2. Checkout do repositório
3. Verificação da presença dos ficheiros essenciais do projeto
4. Configuração do Node.js 20
5. Instalação de dependências (`npm install`)
6. Lint ao `index.html` com `HTMLHint`
7. Execução dos unit tests com Jest (11 testes, 4 suites)
8. Validação do `vercel.json`
9. Verificação de sintaxe do JavaScript
10. Deploy automático pelo Vercel após aprovação do pipeline

A aplicação encontra-se disponível em: <https://chromamood.vercel.app/>

CI/CD - ChromaMood

ci.yml

Filter workflow runs

...

9 workflow runs

Event

Status

Branch

Actor

✓

revert: voltar ao llama-3.3-70b com prompt few-shot

CI/CD - ChromaMood #9: Commit [a7844ce](#) pushed by [rodrigo09malheiro](#)

main

🕒 24 minutes ago

🕒 24s

...

✓

feat: paletas baseadas em categorias emocionais para consistência vi...

CI/CD - ChromaMood #8: Commit [641088f](#) pushed by [rodrigo09malheiro](#)

main

🕒 27 minutes ago

🕒 26s

...

✓

fix: trocar modelo Groq para gemma2-9b-it para melhor coerência d...

CI/CD - ChromaMood #7: Commit [22d6771](#) pushed by [rodrigo09malheiro](#)

main

🕒 30 minutes ago

🕒 24s

...

Figure 4 - Workflows runs

Production Deployment

Repository

Instant Rollback

Visit

ChromaMood

Carra paletas de cor com IA a partir de uma descrição

Descreva um mood... ou uma imagem a partir de

Gerar Paleta

Histórico de Paletas

Deployment

chromamood-87h72thjt-rodrigo09malheiros-projects.vercel.app

Domains

chromamood.vercel.app

Status

Created

Ready

25m ago by rodrigo09malheiro

Source

main

a7844ce revert: voltar ao llama-3.3-70b com prompt few-shot

Deployment Settings

4 Recommendations

To update your Production Deployment, push to the main branch.

Deployments

Figure 5 - Deploy do Vercel

5. Prints e Evidências

5.1 Aplicação em Funcionamento

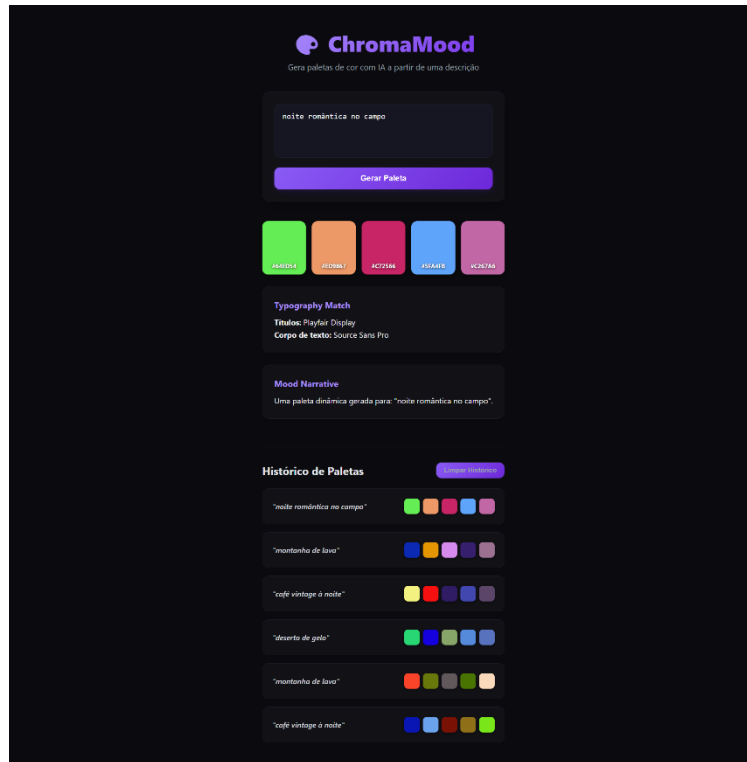


Figure 6 - Interface do Chromamood

5.2 GitHub

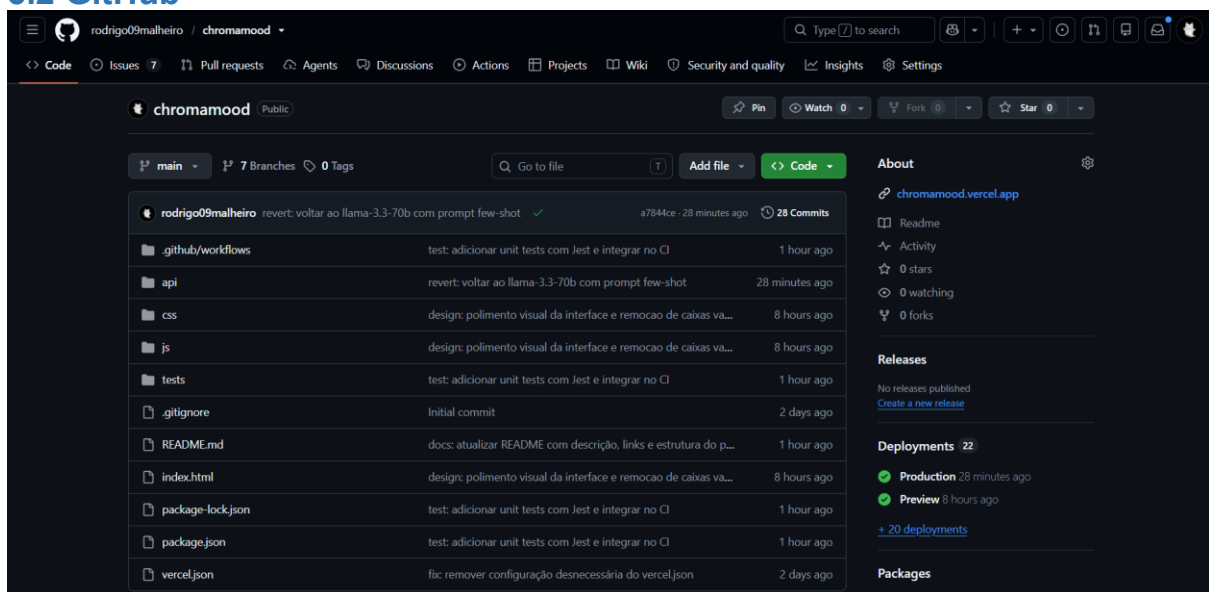


Figure 7 - Página inicial do repositório

main	All users	All time
Commits on Jun 18, 2026		
revert: voltar ao llama-3.3-70b com prompt few-shot		
rodrigo09malheiro committed 17 minutes ago - 3 / 3		
a7844ce		
feat: paletas baseadas em categorias emocionais para consistência visual		
rodrigo09malheiro committed 20 minutes ago - 3 / 3		
641088f		
fix: trocar modelo Groq para gemma2-9b-it para melhor coerência de paletas		
rodrigo09malheiro committed 23 minutes ago - 3 / 3		
22d6771		
feat: melhorar prompt com few-shot examples para paletas coerentes		
rodrigo09malheiro committed 25 minutes ago - 3 / 3		
acaccf4		
feat: melhorar prompt do sistema para paletas mais coerentes		
rodrigo09malheiro committed 30 minutes ago - 3 / 3		
e474ad9		
docs: atualizar README com descrição, links e estrutura do projeto		
rodrigo09malheiro committed 45 minutes ago - 3 / 3		
02b73a4		
test: adicionar unit tests com Jest e integrar no CI		
rodrigo09malheiro committed 50 minutes ago - 3 / 3		
22d1b23		

Figure 8 - Lista de Commits

Overview	Yours	Active	Stale	All	New branch
Search branches...					
Branch	Updated	Check status	Behind	Ahead	Pull request
feat/ui-form-swatches-3	8 hours ago	2 / 2	11	0	#14
feature/integracao-groq	2 days ago	2 / 2	22	0	#12
feature/cicd-vercel	2 days ago	2 / 2	20	0	#10
feature/estrutura-base	2 days ago		26	0	#7
eduardo	2 days ago		27	0	
rodrigo	2 days ago		27	0	

Figure 9 - Lista de Branches

0 Open 8 Closed		Author	Label	Projects	Milestones	Reviews	Assignee	Sort
☐	Feat/UI form swatches 3 ✓	#14 by eduooliveira31 (Collaborator)	was merged 8 hours ago	Review required				1
☐	feat: formulario e swatches de cor com copia hex concluidos #3 ✓	#13 by eduooliveira31 (Collaborator)	was merged yesterday	Review required				1
☐	fix: atualizar modelo Groq para llama-3.3-70b-versatile ✓	#12 by rodrigo09malheiro (Owner)	was merged 2 days ago	Review required				1
☐	fix: melhorar parsing da resposta da Groq ✓	#11 by rodrigo09malheiro (Owner)	was merged 2 days ago	Review required				1
☐	fix: simplificar configuração vercel.json ✓	#10 by rodrigo09malheiro (Owner)	was merged 2 days ago	Review required				1
☐	feat: configuração CI/CD e deploy Vercel	#9 by rodrigo09malheiro (Owner)	was merged 2 days ago	Review required				1
☐	feat: integração com API Groq	#8 by rodrigo09malheiro (Owner)	was merged 2 days ago	Review required				1
☐	feat: estrutura base do projeto	#7 by rodrigo09malheiro (Owner)	was merged 2 days ago	Review required				1

Figure 10 - Lista de PR's (Pull Requests)

5.3 Scrum Board

ChromaMood Scrum Board					Add status update	Insights	Workflows	...
View 1	+ New view	Filter by keyword or by field			View			
Product Backlog 3 This item hasn't been started - chromamood #6 [BUG] teste de reporte de bug - eduardo - chromamood #20 [FEAT] Exportar paleta como imagem PNG - chromamood #21 [FEAT] Modo escuro / claro	Sprint 1 3 - chromamood #17 [DOCS] Escrever relatório final do projeto - chromamood #18 [TEST] Adicionar unit tests com Jest ao CI - chromamood #19 [DOCS] Atualizar README com link e estrutura	Em Progresso 1 This has been completed chromamood #15 [DOCS] Relatório final - rodrigo	Em Revisão 1 chromamood #16 [TEST] Unit tests com Jest - rodrigo	Conclusão 5 - chromamood #5 [FEAT] Pipeline CI/CD e deploy no Vercel - rodrigo #9 - chromamood #2 [FEAT] Integração com API Groq - rodrigo #8 - chromamood #1 [FEAT] Estrutura base do projeto - rodrigo #7 - chromamood #4 [FEAT] Histórico de paletas (localStorage) - eduardo - chromamood #3 [FEAT] UI - Formulário e swatches de cor - eduardo				

Figure 11 - Scrum Board

5.4 CI/CD e Deploy



Figure 12 - Workflow runs

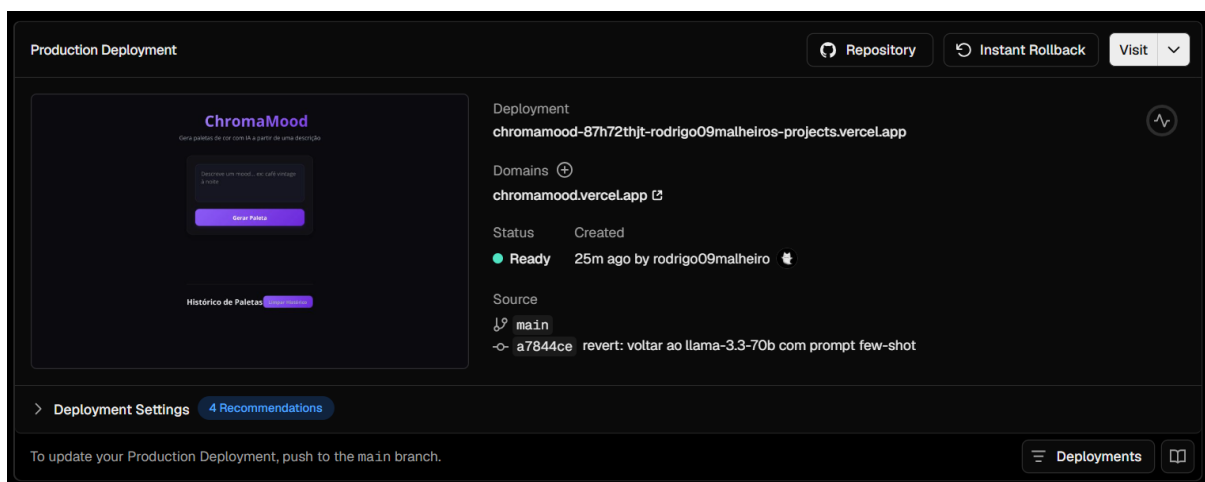


Figure 13 - Deploy do Vercel

6. Conclusão e Análise Crítica

6.1 Processo e Metodologias de Desenvolvimento

O desenvolvimento do ChromaMood seguiu uma abordagem Agile/Scrum adaptada à dimensão do projeto e da equipa. A utilização de feature branches com Pull Requests promoveu uma revisão sistemática do código antes da integração, reduzindo a introdução de bugs na branch principal. A gestão das tarefas no GitHub Projects permitiu manter visibilidade sobre o estado do projeto em todo o momento, com uma distinção clara entre trabalho planeado, em curso e concluído.

A adoção de práticas de CI/CD desde cedo no projeto revelou-se particularmente vantajosa: cada push para main é automaticamente validado pelo pipeline antes do deploy, garantindo que a versão em produção é sempre funcional e testada. Este nível de automação seria difícil de justificar num projeto individual de pequena dimensão, mas o exercício demonstrou claramente o seu valor em contexto colaborativo.

6.2 Potencial das Ferramentas de IA na Geração e Automatização

A utilização do Claude e do Google Gemini como assistentes de vibe coding acelerou significativamente o desenvolvimento. Tarefas que normalmente exigiriam pesquisa e escrita manual — como a configuração do workflow de GitHub Actions, a criação dos unit tests com Jest, ou a integração com a API da Groq — foram concluídas em minutos através de prompts bem estruturados.

A IA demonstrou particular utilidade em três áreas: geração de código boilerplate, depuração de erros (especialmente no parsing da resposta da Groq) e geração de testes unitários. O Claude revelou-se mais preciso para tarefas de código específico, enquanto o Gemini foi mais útil para pesquisa conceptual e explicações de alto nível.

No entanto, o código gerado requereu sempre revisão humana. A IA nem sempre produziu soluções otimizadas à primeira, e foi necessário um processo iterativo de refinamento por prompts para atingir a qualidade desejada.

6.3 Limitações Encontradas

A principal limitação técnica foi a inconsistência das paletas geradas pelo modelo LLM. Apesar de múltiplas iterações no system prompt — incluindo técnicas de few-shot prompting e chain-of-thought — o modelo llama-3.3-70b-versatile na Groq não produz consistentemente paletas emocionalmente coerentes com o mood descrito. Esta limitação é inerente à natureza probabilística dos LLMs e à dificuldade em codificar princípios subjetivos de design em instruções textuais.

A nível de processo, a dimensão reduzida da equipa limitou a profundidade da prática Scrum — reuniões de sprint planning e retrospectiva foram informais, e a distribuição de tarefas foi mais sequencial do que verdadeiramente paralela.

6.4 Vantagens do Trabalho Colaborativo

A divisão de responsabilidades entre os dois elementos permitiu paralelizar o desenvolvimento: enquanto a integração com a API e o CI/CD foram desenvolvidos por Rodrigo, o Eduardo focou-se na experiência do utilizador — formulário, swatches interativos e histórico de paletas com localStorage. Esta separação de responsabilidades, suportada pelo modelo de feature branches, minimizou conflitos e permitiu um fluxo de trabalho eficiente.

O uso de Pull Requests como mecanismo de revisão de código introduziu uma camada de qualidade que, mesmo numa equipa de dois elementos, demonstrou valor — identificando inconsistências e promovendo discussão sobre as decisões de implementação.

6.5 Impacto do CI/CD no Desenvolvimento

A implementação do pipeline de CI/CD teve um impacto positivo imediato na qualidade e confiança do processo de desenvolvimento. A execução automática dos 11 testes unitários a cada push criou uma rede de segurança que detetaria rapidamente eventuais regressões. O deploy automático pelo Vercel eliminou completamente o processo manual de publicação, reduzindo o tempo entre o desenvolvimento e a disponibilização em produção a segundos.

Esta experiência demonstrou que a combinação de vibe coding com IA e práticas rigorosas de CI/CD não são mutuamente exclusivas — pelo contrário, complementam-se: a IA acelera a geração de código, e o CI/CD garante que esse código é continuamente validado antes de chegar à produção.

7. Bibliografia

Anthropic. (2024). Claude. <https://www.anthropic.com/claude>

Fowler, M., & Highsmith, J. (2001). The agile manifesto. *Software Development*, 9(8), 28–35.

GitHub. (2024). GitHub Actions documentation. <https://docs.github.com/en/actions>

Google. (2024). Gemini. <https://gemini.google.com>

Groq. (2024). Groq API documentation. <https://console.groq.com/docs>

Meta AI. (2024). Llama 3. <https://ai.meta.com/llama/>

Schwaber, K., & Sutherland, J. (2020). The Scrum guide. <https://scrumguides.org/scrum-guide.html>

Vercel. (2024). Vercel documentation. <https://vercel.com/docs>